Affective Computing – Programming Assignment 4

Objective

Train a multimodal emotion recognition system using the CSC Roihu supercomputer and the PyTorch machine learning library. You are given a code project where some critical components are missing. Your task is to write these missing components and run the training.

You must also write a short report where you answer the questions at the end of this document.

Return your code as a zip file and your report as pdf file to Moodle.

Dataset

The dataset used in this programming assignment is RAVDESS (https://huggingface.co/datasets/mteb/RAVDESS_AV). The dataset contains video and audio data of actors speaking in different emotions.

You do not need to download the dataset yourself. The dataset can be found from Roihu scratch directory (/scratch/project_2020210/dataset/). This dataset path is hard-coded to the code files so you do not need to worry about loading the data.

Model

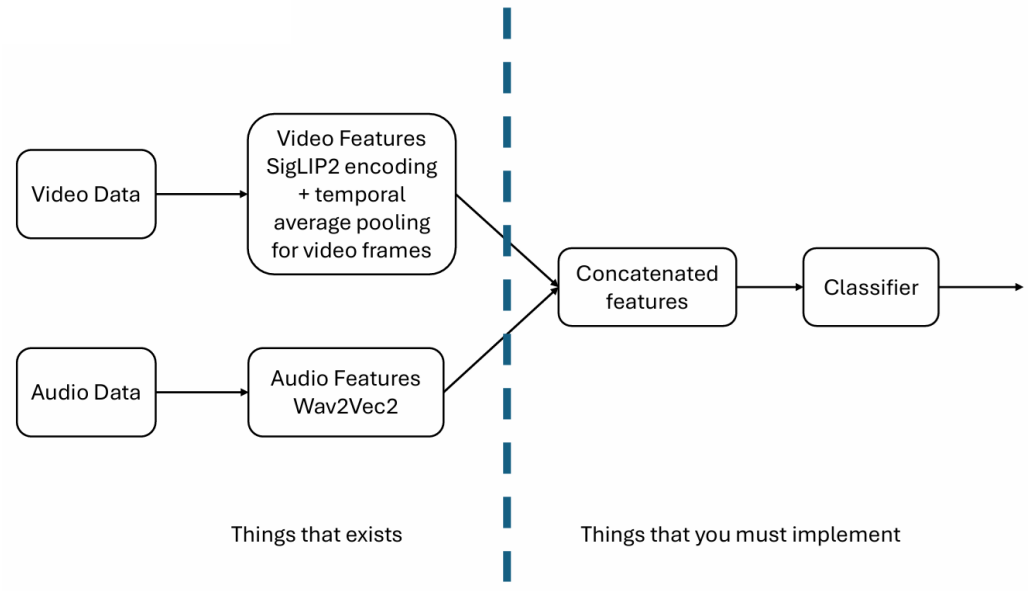


Figure 1 Block diagram of the model. The audio and video features are concatenated and used as an input to a neural network classifier.

As a video encoder we use the SigLIP2 model (for details, see https://huggingface.co/docs/transformers/model_doc/siglip2).

Audio features are computed using Wav2Vec (for details, see https://huggingface.co/facebook/wav2vec2-base).

Both these models have frozen parameters and they are not optimized during the training.

CSC Roihu access

The course has a CSC project. Here is an invitation that you must follow: https://my.csc.fi/projects/invitation/2106225a-ae85-448a-ab87-4fd51f68f6f2

When you have been accepted to the project, you can start working.

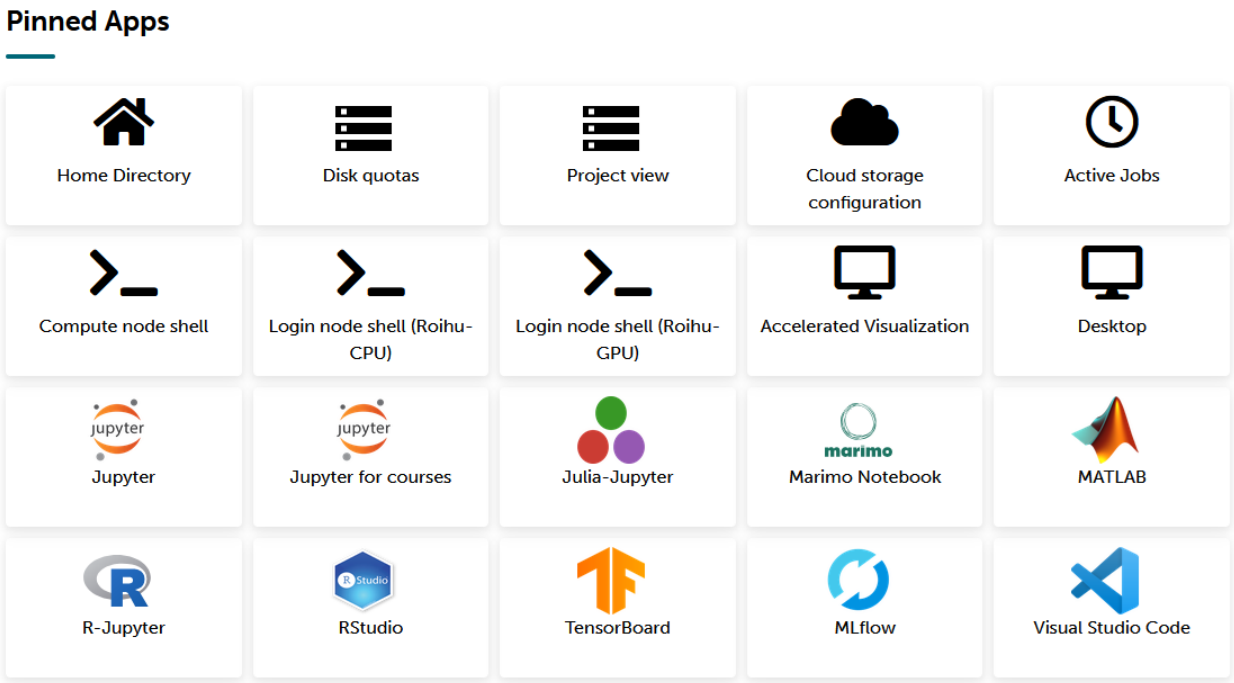


Figure 2 Roihu web ui.

The easiest way to access the Roihu machine is through https://www.roihu.csc.fi.

You can upload the code files to your home directory with the Home Directory app (See, Figure 2). You can also download the completed work back to your own machine using this app too.

The Login node shell (Roihu-GPU) will give you a shell that you can use to work on this assignment.

Extract the exercise_4.zip file to your CSC home directory.

Task 1.1

The slurm script file (scripts/run.sh) is missing the command that starts the training. Look at the train.py file and add the training command to the script.

The command should look like this:

srun python <something>

Run the training using the --subject-independent parameter.

Task 1.2

Create the classifier. See the av_emotion.py file and the AVEmotionModel class. The classifier must be implemented on this line:

classifier = #Your classifier here

The classifier you implement needs an input layer of the size self.video_dim + self.audio_dim. The output size should be equal to the number of classes (num_classes). What happens between these layers is up to you.

Note that in the AVEmotionModel constructor you get dropout and hidden_dim parameters. You are free to use these or modify them in your classifier.

Task 1.3

Concatenate the audio and video features, run the classifier and return the result.

In AVEmotionModel class (av_emotion.py), see the forward method. You are given the audio and video feature encodings. You must concatenate the encodings and call your classifier and return the result.

Task 1.4

Train your model. You must get at least 60% validation accuracy to pass this task. See the slurm output and find the vall_acc value.

The training can be started with the command (from the directory where you extracted the code files):

sbatch scripts/train.sh

By default the slurm job will write its output to out_* and err_* text files in ./slurm_log directory. The model output is written in the out_* file. This includes the validation accuracy among many other things. The errors will be written in the err_* file.

Question 1

What were your validation accuracies over the epochs?

Question 2

What emotions had the best and worst F1 score?

Question 3

Attach your confusion matrix to your report. If you cannot make it look nice as a text, please take a screenshot from your text editor.

Implementation hints

- If there is something wrong with your code and it does not seem to be running, check the slurm_log directory and see the err_* file.
- When developing your solution, use the --limit parameter to limit the number of data samples in training. Also set the --epochs to 1 first. This will make the training go faster, and you can test that your code works.
- When your code is complete, run it with gpumedium partition (gputest is not meant to be used for heavy computation), without the --limit parameter and a higher number of epochs.
- The classifier you implement does not need to be big in terms of parameters to reach the minimum required validation performance. A couple fully connected layers should do it.
- The number of code lines this programming assignment requires is very low. If you find yourself writing hundreds of lines of code, you are doing something wrong.
- You do not get any extra points if you get a result over the minimum required.

Help

See the CSC website for more information on the slurm files:

https://docs.csc.fi/fi/

Pytorch documentation:

https://docs.pytorch.org/docs/2.14/index.html